

 Generate

code to tokenize text for examples and more



Close

< 1 of 1 >
 

[Use code with caution](#)

prompt: code to tokenize text for examples and more

```
import nltk
nltk.download('punkt')
from nltk.tokenize import word_tokenize
```

```
text = "This is an example sentence. Tokenization is cool!"
```

```
# Tokenize the text into words
tokens = word_tokenize(text)
```

```
print(tokens)
```

```

Word Tokenization (nltk.word_tokenize): ['This', 'is', 'an', 'example', 'sentence', '.', 'Tokenization', 'is', 'cool', '!']
String Split (str.split): ['This', 'is', 'an', 'example', 'sentence.', 'Tokenization', 'is', 'cool!']
Differences:
Word_tokenize handles punctuation differently than str.split.
At index 4:
word_tokenize: sentence
str.split: sentence.
At index 5:
word_tokenize: .
str.split: Tokenization
At index 6:
word_tokenize: Tokenization
str.split: is
At index 7:
word_tokenize: is
str.split: cool!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!

```

 Generate

code to show the defreenece between split and toknize



Close

< 1 of 1 >
 

[Use code with caution](#)

prompt: code to show the defreenece between split and toknize

```
import nltk
from nltk.tokenize import word_tokenize
```

```
nltk.download('punkt')
```

```
text = "This is an example sentence. Tokenization is cool!"
```

```
# Tokenize the text into words using word_tokenize
tokens = word_tokenize(text)
```

```
# Split the text into words using the split function
split_words = text.split()
```

```
print("Word Tokenization (nltk.word_tokenize):", tokens)
print("String Split (str.split):", split_words)
```

```

# Compare the outputs to highlight differences
print("\nDifferences:")
if tokens != split_words:
    print("Word_tokenize handles punctuation differently than str.split.")
    for i in range(min(len(tokens), len(split_words))):
        if tokens[i] != split_words[i]:
            print(f"At index {i}:")
            print(f"word_tokenize: {tokens[i]}")
            print(f"str.split: {split_words[i]}")

```

```

Word Tokenization (nltk.word_tokenize): ['This', 'is', 'an', 'example', 'sentence', '.', 'Tokenization', 'is', 'cool', '!']
String Split (str.split): ['This', 'is', 'an', 'example', 'sentence.', 'Tokenization', 'is', 'cool!']
Differences:
Word_tokenize handles punctuation differently than str.split.
At index 4:
word_tokenize: sentence
str.split: sentence.
At index 5:
word_tokenize: .
str.split: Tokenization
At index 6:
word_tokenize: Tokenization
str.split: is
At index 7:
word_tokenize: is
str.split: cool!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!

```

 Generate

code to print stop words



Close

```
# prompt: code to print stop words
```

```
import nltk
from nltk.corpus import stopwords

nltk.download('stopwords')

# Get the English stop words
stop_words = set(stopwords.words('english'))

print(stop_words)
print(len(stop_words))
```

```
{ 'our', 'own', 'below', 'at', 'doesn', 'once', 'when', 'or', 'hadn', 'has', 'such', 'aren', 'me', 'shan't', 'very', 'mightn't', 'yo
179
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
```

```
# prompt: code to print stop words
```

```
import nltk
from nltk.corpus import stopwords

nltk.download('stopwords')

# Get the English stop words
stop_words = set(stopwords.words('arabic'))

print(stop_words)
print(len(stop_words))
```

```
{ 'إياهم', 'كلّ', 'هم', 'هي', 'مايو', 'سبعة', 'عسى', 'ذلكم', 'كانون', 'أغسطس', 'إيانا', 'ثلاثين', 'أ', 'كاد', 'قد', 'لكيلا', 'أفريل', 'ى', 'عشرين', 'ليسوا'
701
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
```

Generate	code to make my stopwords	Close
1 of 1 Use code with caution		

```
# prompt: code to make my stopwords
```

```
my_stop_words = ['my', 'custom', 'stop', 'words']
```

Generate	code to remove stop words using if	Close
1 of 1 Use code with caution		

```
# prompt: code to remove stop words using if
```

```
nltk.download('stopwords')

# Get the English stop words
stop_words = set(stopwords.words('english'))

# Example sentence
text = "This is an example sentence, showing how to remove stop words."

# Tokenize the sentence into words
tokens = word_tokenize(text)

# Remove stop words using an if statement
filtered_tokens = []
for token in tokens:
    if token.lower() not in stop_words:
        filtered_tokens.append(token)

print("Original tokens:", tokens)
print("Tokens after removing stop words:", filtered_tokens)
```

```
Original tokens: ['This', 'is', 'an', 'example', 'sentence', ',', 'showing', 'how', 'to', 'remove', 'stop', 'words', '.']
Tokens after removing stop words: ['example', 'sentence', ',', 'showing', 'remove', 'stop', 'words', '.']
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
```

 Generate

code to show deffrence between steaming and limitization by spacy



Close

< 1 of 1 >
 

[Use code with caution](#)

```
# prompt: code to show deffrence between steaming and limitization by spacy
```

```
!pip install spacy
!python -m spacy download en_core_web_sm
```

```
import spacy
```

```
nlp = spacy.load("en_core_web_sm")
```

```
text = "running runs ran runner"
```

```
doc = nlp(text)
```

```
# Stemming (using a rule-based approach)
print("Stemming:")
for token in doc:
    print(token.text, "->", token.lemma_)
```

```
# Lemmatization (using a dictionary-based approach)
print("\nLemmatization:")
for token in doc:
    print(token.text, "->", token.lemma_)
```

```
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.10/dist-packages (from markdown-it-py>=2.2.0->rich>=10.11.0->
Collecting en-core-web-sm==3.7.1
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.7.1/en_core_web_sm-3.7.1-py3-none-any.
12.8/12.8 MB 67.0 MB/s eta 0:00:00
Requirement already satisfied: spacy<3.8.0,>=3.7.2 in /usr/local/lib/python3.10/dist-packages (from en-core-web-sm==3.7.1) (3.7.6
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.2->
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.2->
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.2->en-core-
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.2->en-core-
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.2->en-core-
Requirement already satisfied: thinc<8.3.0,>=8.2.2 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.2->en-core-
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.2->en-core-
Requirement already satisfied: srsly<3.0.0,>=2.4.3 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.2->en-core-
```

```
run -> run
runner -> runner
*****
```